

JML-Lab

Joiner · Mover · Leaver — Automation mit Microsoft Graph PowerShell

Praxis-Cheatsheet · erstellt August 2026 · Alexander Frank · alle Befehle in einem echten Entra-Tenant getestet

Dieses Dokument fasst die acht Kernbefehle zusammen, mit denen sich der komplette Identity-Lifecycle in Microsoft Entra ID automatisieren lässt — vom Anlegen eines Mitarbeiters (Joiner) über den Abteilungswechsel (Mover) bis zum Offboarding (Leaver). Jeder Befehl steht mit Syntax, Zweck, den wichtigsten Parametern und den typischen Fehlern. Am Ende folgt eine Schnellreferenz zum Auswendiglernen.

1. Connect-MgGraph **BASIS**

Was: Baut die Verbindung zwischen PowerShell und Entra ID auf und fordert dabei genau die Rechte (Scopes) an, die das Skript braucht.

Wann: Ganz am Anfang jedes Skripts — ohne Verbindung läuft kein einziger Graph-Befehl.

```
# Verbindung mit den Rechten fuer den kompletten JML-Prozess
Connect-MgGraph -Scopes "User.ReadWrite.All","Group.ReadWrite.All","Directory.ReadWrite.All"

# Verbindung pruefen: wer bin ich, welcher Tenant, welche Scopes?
Get-MgContext

# Session sauber beenden
Disconnect-MgGraph
```

Wichtige Scopes:

- **User.ReadWrite.All** — Benutzer lesen und ändern
- **Group.ReadWrite.All** — Gruppenmitgliedschaften verwalten
- **Directory.ReadWrite.All** — Verzeichnisobjekte ändern
- **Organization.Read.All** — Lizenzen (SKUs) auslesen

Least Privilege: Immer nur die Scopes anfordern, die der aktuelle Schritt wirklich braucht. Ein reines Lese-Skript nimmt **User.Read.All**, kein Schreibrecht.

Häufige Fehler: „Timeout“ → mit **-UseDeviceAuthentication** arbeiten · „not recognized“ → Modul fehlt (**Install-Module Microsoft.Graph**) · falscher Account → **Disconnect-MgGraph** und mit dem richtigen Firmenkonto neu verbinden.

2. Get-MgUser **LESEN**

Was: Benutzer auslesen und suchen. Die Grundlage für jeden weiteren Schritt — vor jedem Ändern oder Löschen wird erst gelesen.

Wann: Immer, wenn ein Benutzer gefunden werden muss, bevor etwas mit ihm passiert.

```
# Die ersten 5 Benutzer mit ausgewählten Feldern
Get-MgUser -Top 5 -Property DisplayName,UserPrincipalName,Id,AccountEnabled

# Nach E-Mail suchen (EMPFOHLEN, weil eindeutig)
Get-MgUser -Filter "userPrincipalName eq 'max.mustermann@firma.de'"

# Teiltreffer per startsWith
Get-MgUser -Filter "startsWith(userPrincipalName, 'max')"
```

Eindeutigkeit: Immer nach **UserPrincipalName** oder **Id** filtern, nie nach **DisplayName** — Namen können mehrfach vorkommen, die E-Mail nie.

3. New-MgUser JOINER

Was: Legt einen neuen Benutzer im Tenant an — mit Name, E-Mail und temporärem Passwort.

Wann: Onboarding — HR stellt einen neuen Mitarbeiter ein.

```
$newUser = @{
    DisplayName      = "Max Mustermann"
    MailNickname     = "max.mustermann"
    UserPrincipalName = "max.mustermann@firma.de"
    AccountEnabled   = $true
    PasswordProfile  = @{
        ForceChangePasswordNextSignIn = $true
        Password                       = "TempPassword123!@#"
    }
}
$createdUser = New-MgUser @newUser
$createdUser | Select-Object DisplayName, UserPrincipalName, Id
```

Pflichtfelder: DisplayName · MailNickname (eindeutig) · UserPrincipalName (eindeutig) · PasswordProfile

Passwort-Sicherheit: Das Temp-Passwort verlässt nie E-Mail oder Chat. Über **ForceChangePasswordNextSignIn = \$true** muss der Benutzer beim ersten Login sein eigenes setzen.

4. Update-MgUser MOVER

Was: Ändert Attribute eines bestehenden Benutzers, ohne ihn neu anzulegen.

Wann: Abteilungswechsel, Beförderung, Standortwechsel.

```
$userId = (Get-MgUser -Filter "userPrincipalName eq 'max.mustermann@firma.de'").Id

Update-MgUser -UserId $userId -BodyParameter @{
    Department      = "Marketing"
    JobTitle         = "Senior Manager"
    OfficeLocation   = "Berlin"
}
```

Merke: Änderungen laufen über **-BodyParameter @{ ... }**. Es werden nur die genannten Felder überschrieben (PATCH), alles andere bleibt unverändert.

5. New-MgGroupMember JOINER

Was: Fügt einen Benutzer einer Gruppe hinzu — die Basis für Zugriffe und Conditional-Access-Policies.

Wann: Beim Onboarding (Gruppen zuweisen) und beim Wechsel (Gruppe tauschen).

```
$groupId = (Get-MgGroup -Filter "displayName eq 'GRP-INTUNE-PILOT-USERS'").Id
$userId  = (Get-MgUser -Filter "userPrincipalName eq 'max.mustermann@firma.de'").Id

New-MgGroupMember -GroupId $groupId -DirectoryObjectId $userId

# Mitglieder prüfen
Get-MgGroupMember -GroupId $groupId | Select-Object DisplayName, Id
```

Gruppentypen:

- **{}** — Sicherheitsgruppe (Conditional Access, Policies)
- **{Unified}** — Microsoft-365-Gruppe (Teams, SharePoint)
- **{DynamicMembership}** — Regelbasierte Mitgliedschaft (nicht manuell)

Erwartetes Verhalten: „One or more added object references already exist“ heißt schlicht, dass der Benutzer bereits Mitglied ist — kein echter Fehler.

6. Set-MgUserLicense **JOINER**

Was: Weist einem Benutzer eine Microsoft-365-Lizenz zu. Ohne Lizenz kann er sich anmelden, aber nichts nutzen.

Wann: Direkt nach dem Anlegen, damit der Mitarbeiter arbeiten kann.

```
# Verfügbare Lizenzen (SKUs) auflisten
Get-MgSubscribedSku | Select-Object SkuPartNumber, SkuId

$licenseAssignment = @{
    AddLicenses      = @( @{ SkuId = "<SkuId>" } )
    RemoveLicenses   = @()
}
Set-MgUserLicense -UserId $userId -BodyParameter $licenseAssignment
```

Häufige SKUs: ENTERPRISEPACK (E3) · ENTERPRISEPREMIUM (E5) · STANDARDPACK (Business Standard)

Fehler „does not correspond to a valid company License“: Die SKU-ID gehört nicht zum Tenant oder der Tenant hat keine Lizenzen gekauft (typisch für Dev-Tenants). Struktur bleibt identisch.

7. Update-MgUser — Deaktivieren **LEAVER 1**

Was: Setzt **AccountEnabled = \$false** — der Benutzer kann sich nicht mehr anmelden, die Daten bleiben aber erhalten.

Wann: Erster Offboarding-Schritt, sobald ein Mitarbeiter das Unternehmen verlässt.

```
$userId = (Get-MgUser -Filter "userPrincipalName eq 'max.mustermann@firma.de'").Id

Update-MgUser -UserId $userId -BodyParameter @{ AccountEnabled = $false }

Get-MgUser -UserId $userId -Property DisplayName, AccountEnabled
```

Deaktivieren vs. Löschen: Deaktivieren ist reversibel, die Daten bleiben (Standard 90 Tage Aufbewahrungsfenster im Prozess). Erst danach folgt das endgültige Löschen.

8. Remove-MgUser **LEAVER 2**

Was: Löscht den Benutzer endgültig aus dem Tenant.

Wann: Nach Ablauf der Aufbewahrungsfrist, als Abschluss des Leaver-Prozesses.

```
$userId = (Get-MgUser -Filter "userPrincipalName eq 'max.mustermann@firma.de'").Id

Remove-MgUser -UserId $userId

# Kontrolle: keine Ausgabe = erfolgreich gelöscht
Get-MgUser -Filter "userPrincipalName eq 'max.mustermann@firma.de'"
```

Nicht reversibel: Gelöschte Objekte liegen noch 30 Tage im Papierkorb, danach sind sie weg. In Produktion immer eine Sicherheitsabfrage (**Read-Host**) vorschalten.

Kompletter Workflow

Die drei Prozesse als zusammenhängende Skript-Bausteine

Joiner — neuer Mitarbeiter

```
$user = New-MgUser @{
    DisplayName = "Max Mustermann"; MailNickname = "max.mustermann"
    UserPrincipalName = "max.mustermann@firma.de"; AccountEnabled = $true
    PasswordProfile = @{ ForceChangePasswordNextSignIn = $true; Password = "TempPassword123!@#" }
}
$groupId = (Get-MgGroup -Filter "displayName eq 'GRP-INTUNE-PILOT-USERS'").Id
New-MgGroupMember -GroupId $groupId -DirectoryObjectId $user.Id
Set-MgUserLicense -UserId $user.Id -BodyParameter @{ AddLicenses = @(@{SkuId="<SkuId>"});
RemoveLicenses = @() }
```

Mover — Abteilungswechsel

```
$user = Get-MgUser -Filter "userPrincipalName eq 'max.mustermann@firma.de'"
Update-MgUser -UserId $user.Id -BodyParameter @{ Department = "Marketing"; JobTitle = "Senior
Manager" }
Remove-MgGroupMember -GroupId $oldGroupId -DirectoryObjectId $user.Id
New-MgGroupMember -GroupId $newGroupId -DirectoryObjectId $user.Id
```

Leaver — Mitarbeiter verlässt das Unternehmen

```
$user = Get-MgUser -Filter "userPrincipalName eq 'max.mustermann@firma.de'"
Update-MgUser -UserId $user.Id -BodyParameter @{ AccountEnabled = $false } # sofort deaktivieren
# ... Aufbewahrungsfrist abwarten ...
Remove-MgUser -UserId $user.Id # danach endgültig
loeschen
```

Schnellreferenz — die wichtigsten Graph-Befehle

Zum Auswendiglernen

#	Befehl	Aktion	Kern-Syntax
1	Connect-MgGraph	Verbindung	-Scopes "User.ReadWrite.All",...
2	Get-MgUser	Lesen / Suchen	-Filter "userPrincipalName eq '...'"
3	New-MgUser	Anlegen (Joiner)	@{ DisplayName; UPN; PasswordProfile }
4	Update-MgUser	Ändern (Mover)	-BodyParameter @{ Department;... }
5	New-MgGroupMember	Gruppe zuweisen	-GroupId ... -DirectoryObjectId ...
6	Set-MgUserLicense	Lizenz	-BodyParameter @{ AddLicenses;... }
7	Update-MgUser	Deaktivieren (Leaver)	@{ AccountEnabled = \$false }
8	Remove-MgUser	Löschen (Leaver)	-UserId \$id (!)

Nützliche Zusatzbefehle

Befehl	Zweck
Get-MgContext	Aktuelle Verbindung, Konto und Scopes prüfen
Get-MgGroup	Gruppen auflisten oder per Filter suchen
Get-MgGroupMember	Mitglieder einer Gruppe anzeigen
Remove-MgGroupMember	Benutzer aus einer Gruppe entfernen
Get-MgSubscribedSku	Verfügbare Lizenzen (SKUs) auslesen
Disconnect-MgGraph	Session beenden, Token verwerfen

Interview-Kurzform: Onboarding = New-MgUser → New-MgGroupMember → Set-MgUserLicense. Offboarding = Update-MgUser (AccountEnabled \$false) → Aufbewahrung → Remove-MgUser. Immer nach UPN oder Id filtern, nie nach DisplayName.